

Homigo Care Android Application Blueprint

- [HOMIGO CARE — ANDROID APPLICATION](#)
 - [Complete Native Build Blueprint for an AI Coding Agent \(Manus AI\)](#)
 - [Repository, Infrastructure, Design System, Screen-by-Screen Specification](#)
 - [0. Purpose of This Document & How the Agent Should Use It](#)
 - [1. Repository & Infrastructure Setup \(execute in this order\)](#)
 - [1.1 GitHub Repository](#)
 - [1.2 Firebase Project \(via Firebase CLI MCP\)](#)
 - [1.3 Supabase Project \(via Supabase MCP\)](#)
 - [1.4 Cloudinary \(via Cloudinary MCP\)](#)
 - [1.5 Admin Panel Connection \(Vercel MCP — read/verify only from this repo's side\)](#)
 - [1.6 Secrets & Environment Configuration](#)
 - [1.7 Founder-Provided Assets — Exact Drop-In Paths](#)
 - [1.8 AI Image-Generation Request Pipeline — 01_ai_images.json](#)
 - [2. Global Design System — Single Source of Truth](#)
 - [3. Technology Stack \(Android — final, authoritative\)](#)
 - [4. Icons — What's Coded vs. What's AI-Generated \(decision rule\)](#)
 - [5. Splash Screen & Video Integration](#)
 - [6. Authentication & OTP Flow \(via the Admin Panel's EmailJS SMTP\)](#)
 - [7. Complete Screen-by-Screen Blueprint](#)
 - [7.1 Shared — Splash, Onboarding, Auth, Role Selection \(9 screens\)](#)
 - [7.2 Patient Flow — 32 Screens \(P-01-P-32\)](#)
 - [7.3 Professional Flow — 15 Screens \(PR-01-PR-15\)](#)
 - [8. App-Wide Navigation Map \(text flow, for NavGraph.kt\)](#)
 - [9. Data Sync Architecture \(Android-side summary\)](#)
 - [10. Build & Release Pipeline](#)
 - [11. Final OA Checklist Before Calling the Build "Done"](#)

HOMIGO CARE — ANDROID APPLICATION

Complete Native Build Blueprint for an AI Coding Agent (Manus AI)

Repository, Infrastructure, Design System, Screen-by-Screen Specification

0. Purpose of This Document & How the Agent Should Use It

This is a **single, self-contained build order** for the **official native Android application** of Homigo Care. It is written for an AI coding agent (Manus AI) that has the following MCP servers already connected and ready to use:

- **GitHub MCP** — create/manage the repository and push code
- **Supabase MCP** — create the project, run schema migrations, manage Row Level Security
- **Firebase CLI MCP** — create the Firebase project and enable services
- **Vercel MCP** — read-only awareness of the separately-hosted Admin Panel project (the Android app is a *client* of it, not a host of it)
- **Cloudinary MCP** — create upload presets/folders for app assets and user documents

This PDF covers the Android application only. The Admin Panel is a separate project/repository, built and deployed independently on Vercel, and is treated here strictly as “the backend the Android app talks to.” Do not build admin screens inside this Android repository.

Founder-provided assets (arrive separately, outside this PDF): the official app logo (PNG/vector), the splash-screen video file, and any additional real photography. Wherever this document says “founder-provided,” the agent should treat that asset as authoritative and drop it into the exact path given in §1.7 — never generate a placeholder for these three items.

Golden consistency rule — read this before writing any screen: every single screen in this app pulls its colors, fonts, spacing, and corner-radius values from **one theme object** (§2). No screen may hardcode a hex color or a font size. If a screen “needs a slightly different shade,” that shade must be added to the shared theme first, then referenced — never inlined. This is the single most important rule in this document, because visual drift between screens was explicitly flagged as unacceptable.

1. Repository & Infrastructure Setup (execute in this order)

1.1 GitHub Repository

1. Using the **GitHub MCP**, create a new repository under the connected account named exactly:

HOMIGO - CARE - ANDROID

Private repository, initialized with an Android .gitignore and an MIT or proprietary LICENSE file (founder's choice — default to "All Rights Reserved" proprietary if not specified).

2. Create this folder structure at the root and commit it before writing any feature code:

```
HOMIGO-CARE-ANDROID/
├── app/
│   ├── src/main/java/pk/homigocare/android/
│   │   ├── core/           # Theme.kt, Shapes.kt, Type.kt, Color.kt, Constants.kt
│   │   ├── data/
│   │   │   ├── firebase/   # Auth, Firestore, FCM wrappers
│   │   │   ├── supabase/   # Postgres repositories via Supabase Kotlin SDK
│   │   │   ├── cloudinary/ # Upload/signed-URL helpers
│   │   │   └── repository/ # Repository pattern combining the above
│   │   ├── domain/        # Use cases, models
│   │   ├── ui/
│   │   │   ├── shared/     # Splash, Onboarding, Auth, RoleSelect
│   │   │   ├── patient/   # All P-xx screens (Compose)
│   │   │   ├── professional/ # All PR-xx screens (Compose)
│   │   │   └── components/ # Shared reusable Compose components (buttons, cards, etc.)
│   │   └── navigation/     # NavGraph.kt
│   ├── src/main/res/
│   │   ├── drawable/      # Vector icons authored in code/XML (Material Symbols style)
│   │   ├── raw/           # splash_video.mp4 (founder-provided)
│   │   ├── mipmap-*/*     # Adaptive app icon (from founder logo)
│   │   └── values/
│   ├── google-services.json # Firebase config (added in §1.2)
│   └── build.gradle.kts
├── assets_pending/
│   └── 01_ai_images.json   # AI image-generation request file – see §1.8
├── .github/workflows/
│   └── android-release.yml # CI: build signed .aab/.apk (see §10)
├── docs/
│   └── this-blueprint.pdf
└── README.md
```

1.2 Firebase Project (via Firebase CLI MCP)

1. Create a new Firebase project: homigo-care-prod (and optionally homigo-care-dev for testing).
2. Enable exactly these services (free Spark plan):
 - **Authentication** → enable **Email/Password**, **Google**, and **Anonymous (Guest)** sign-in methods.
 - **Cloud Firestore** → create in production mode, default region closest to Pakistan (e.g. asia-south1).
 - **Cloud Messaging (FCM)** → no extra setup needed beyond project creation; generate a server key for the Admin Panel to use when sending targeted pushes.
3. Register an **Android app** inside the Firebase project with package name pk.homigocare.android. Download google-services.json and commit it to app/google-services.json.
4. Add the Firebase BoM plus these SDKs to app/build.gradle.kts: firebase-auth, firebase-firestore, firebase-messaging, firebase-analytics (optional).
5. Firestore is used **only** for real-time/live data (per the main architecture document): presence, bookings_live, chat_threads, notifications_feed, booking_status_events. Everything structured/relational goes to Supabase (§1.3). Do not duplicate the same data in both places.

1.3 Supabase Project (via Supabase MCP)

1. Create the Supabase project homigo-care.
2. Run one migration that creates every table listed in the master architecture (users, patients, professionals, role_requirements, addresses, bookings, health_passport, vitals, documents, reviews, payments, payment_accounts, wallet_transactions, assessment_tests, supply_requests, cms_content, access_logs). Table columns follow the field names implied by each screen's content in §7 — the agent should infer standard columns (id, created_at, foreign keys, status enums) consistent with the screen specs.
3. Enable **Row Level Security** on every table. Baseline policy pattern: a patients row is readable/writable only where auth.uid() = user_id; a professionals row similarly; payment_accounts and admin-only tables are readable only by rows where the JWT's role claim is one of the 5 admin roles (these claims are managed from the Admin Panel project, not from this Android app).
4. Add the **Supabase Kotlin SDK** (supabase-kt: Postgrest, Auth, Storage modules as needed) to build.gradle.kts. The Android app authenticates its Supabase session using the **same Firebase ID token**, bridged via a Postgres function/JWT claim mapping set up once at the database level — this keeps one login (Firebase Auth) working across both Firestore and Supabase without asking the user to log in twice.

1.4 Cloudinary (via Cloudinary MCP)

1. Create folders: /homigo/cnic/, /homigo/documents/, /homigo/profile_photos/, /homigo/prescriptions/, /homigo/receipts/, /homigo/app_assets/icons/, /homigo/app_assets/illustrations/, /homigo/app_assets/badges/.
2. Create an **unsigned upload preset** named homigo_public_assets restricted to the app_assets folder only (safe for the app icons/illustrations pipeline in §1.8).
3. Create a **signed** upload preset named homigo_user_documents for CNIC/degree/prescription/receipt uploads — the Android app must request a signature from a secure backend endpoint (a Supabase Edge Function or the Admin Panel's Vercel API) before uploading; never embed the Cloudinary API secret inside the Android app itself.

1.5 Admin Panel Connection (Vercel MCP — read/verify only from this repo's side)

The Android app does **not** host anything on Vercel. It only needs to know, at build time, the base URL of the already-deployed Admin Panel’s API (e.g. `https://admin.homigocare.pk/api`). This base URL is stored in `local.properties/secrets.properties` (see §1.6) as `ADMIN_API_BASE_URL`. Two endpoints on that base URL matter to this Android app: - `POST /api/otp/send` — triggers an OTP/verification email via the Admin Panel’s EmailJS SMTP configuration (see §6). - `POST /api/otp/verify` — verifies the code the user typed. - `GET /api/cms/:key` — fetches editable content (support numbers, About Us text, FAQ, legal pages) that the Admin Panel’s CMS module (see the Admin Panel Blueprint document) controls — the Android app must **never hardcode** support phone numbers, emails, or legal text; always fetch from this endpoint (with a cached fallback for offline viewing).

1.6 Secrets & Environment Configuration

Create `secrets.properties` (git-ignored) with:

```
SUPABASE_URL=...
SUPABASE_ANON_KEY=...
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_UNSIGNED_PRESET=homigo_public_assets
ADMIN_API_BASE_URL=https://admin.homigocare.pk/api
MAPS_PROVIDER=osm # "osm" or "google" – see main architecture doc §7.4
```

Read these into `BuildConfig` fields via `build.gradle.kts` so no secret ever sits in a checked-in Kotlin file. `google-services.json` (not a true secret for a client app) is committed as normal per Firebase convention.

1.7 Founder-Provided Assets — Exact Drop-In Paths

Asset	Path	Notes
App logo (all sizes)	<code>app/src/main/res/mipmap-*/ic_launcher.png</code> + <code>ic_launcher_round.png</code>	Generate the full adaptive-icon mipmap set from the one master PNG the founder provides using Android Studio’s Image Asset Studio — do not redraw the logo
Splash video	<code>app/src/main/res/raw/splash_video.mp4</code>	See §5 for exact playback implementation
This PDF	<code>docs/this-blueprint.pdf</code>	Committed for future reference/other agents working on the same repo

1.8 AI Image-Generation Request Pipeline — `01_ai_images.json`

Whenever a screen in §7 needs a **custom PNG icon or illustration that cannot be built from code** (i.e. not achievable with Material Symbols/vector Compose icons — see the decision rule in §4), the agent does **not** invent or skip the asset. Instead, it **appends an entry** to a single running file at:

```
assets_pending/01_ai_images.json
```

Rules for this file (hard limits): - Minimum 20, maximum 50 image requests per file/batch (the founder generates these through Meta AI in one sitting). If more than 50 assets are ever needed in total, create `02_ai_images.json`, `03_ai_images.json`, etc. — never exceed 50 entries in one file. - Every entry must fully specify: exact subject, **background type** ("`transparent`" for icons/illustrations that sit on varying app backgrounds, or "`solid_white`"/"`solid_brand_teal`" only when the image is meant to be a full-bleed card image), art style (must match the flat vector illustration style already defined in the main brand blueprint — teal #1B6B63/orange #F5823A palette, no photorealism, no text baked in, no real human faces, no third-party logos), and the **exact destination path** inside `app/src/main/res/drawable/` or a `../res/drawable-nodpi/` folder where the founder will place the finished PNG after generating it. - Structure (this is the exact schema the agent must produce):

```
{
  "batch_id": "01",
  "instructions_for_founder": "Paste the 'images' array below into Meta AI (or another image model) exactly as written, one per image. Once generated, save each PNG using its 'filename' value and place it at its 'destination_path'. Then notify the agent so it can wire each image into its screen.",
  "global_style": {
    "art_style": "flat 2D vector illustration, clean line art, minimal shading",
    "palette": ["#1B6B63", "#0D4A44", "#F5823A", "#FFFFFF"],
    "forbidden": ["photorealism", "3D render", "watermarks", "baked-in text", "real human faces/photos", "copyrighted characters or logos"]
  },
  "images": [
    {
      "id": "img_01",
      "filename": "il_onboarding_homecare.png",
      "destination_path": "app/src/main/res/drawable-nodpi/il_onboarding_homecare.png",
      "used_on_screen": "P-02 Onboarding Slide 1",
      "background": "transparent",
      "size_hint": "2048x2048",
      "prompt": "Flat vector illustration of a friendly nurse in teal scrubs kneeling beside an elderly patient in a home l room, warm reassuring mood, minimal line art, teal and orange accents only, transparent background"
    }
  ]
}
```

- After the founder pushes the generated PNGs into the given `destination_path` values and pushes to GitHub, the agent’s job is to reference each file by its Compose painterResource(`R.drawable.il_onboarding_homecare`) (or via Cloudinary URL if

the founder instead uploads through the `homigo_public_assets` preset from \$1.4 — in that case store the returned Cloudinary URL in a small `AppAssets.kt` constants object instead of a local drawable).

- The agent should proactively fill this file with **every** custom illustration/icon the screens in §7 require, batching sensibly by theme (onboarding art, category badges, empty states, achievement badges, role avatars) — mirroring the batching approach already established in the main brand blueprint document, just re-expressed in this per-file JSON schema instead of prose.

2. Global Design System — Single Source of Truth

Create these files **first**, before any screen, and import them everywhere. No screen-level file may redefine a color, font size, or corner radius.

`core/Color.kt`

```
object HomigoColors {
    val PrimaryTeal      = Color(0xFF1B6B63)
    val DarkTeal         = Color(0xFF0D4A44)
    val HeaderEmerald    = Color(0xFF0B5545) // premium header bars only
    val SecondaryOrange  = Color(0xFFF5823A)
    val White            = Color(0xFFFFFFFF)
    val LightGray        = Color(0xFFF5F5F5) // input backgrounds
    val DarkGray         = Color(0xFF333333) // body text
    val MediumGray       = Color(0xFF888888) // placeholder/secondary text
    val ErrorRed         = Color(0xFFE53935) // SOS + error states only
    val SuccessGreen     = Color(0xFF43A047)
    val GoldElite        = Color(0xFFD4AF37) // Elite badge only
    val SilverTopRated   = Color(0xFFC0C0C0) // Top Rated badge only
}
```

`core/Shapes.kt` — Cards: `RoundedCornerShape(15.dp)` with a subtle flat elevation (2–4dp, no heavy drop shadows). Buttons & input fields: `RoundedCornerShape(25.dp)` (fully rounded pill). Emergency/SOS button: `CircleShape`, always `ErrorRed`, with a pulsing scale animation (`InfiniteTransition`, 0.95↔1.05 scale, 1s loop).

`core/Type.kt` — Font family: **Google Sans / Roboto** (Android system default is acceptable and is the “official Android” choice — do not import a custom font that fights Material 3 defaults). Urdu text anywhere in the UI switches to **Noto Nastaliq Urdu** (bundled as a font resource) automatically based on the active locale. - H1: 22sp Bold · H2: 18sp SemiBold · Body: 14sp Regular · Small/Caption: 12sp Regular · Button label: 16sp Bold.

`core/Theme.kt` — wrap the whole app in one `MaterialTheme` built from the tokens above, using **Material 3 (Material You)**. Light mode only — do **not** implement a dark color scheme for v1 (explicit founder decision).

Spacing scale (use consistently, never a random dp value): 4, 8, 12, 16, 24, 32 dp. Screen horizontal padding is always 16.dp. Minimum tap target 48.dp × 48.dp on every icon button/list row (mobile-safe rule).

3. Technology Stack (Android — final, authoritative)

Layer	Choice	Reason
Language	Kotlin (100%, no Java)	Official modern Android language
UI	Jetpack Compose + Material 3	Official declarative UI toolkit — gives the genuine “real Android app” feel required
Architecture	MVVM + Clean Architecture (data / domain / ui layers, Repository pattern)	Testable, scalable, standard for production
Async	Kotlin Coroutines + Flow	Native async, integrates directly with Compose State/collectAsState
DI	Hilt	Google’s official dependency-injection library
Navigation	Navigation-Compose	Official nav graph, type-safe routes
Local cache	Room	Offline booking history, cached Health Passport, cached CMS content for offline support-info viewing
Networking	Retrofit + OkHttp	Calls to the Admin Panel’s Vercel API endpoints (OTP, CMS)
Realtime backend	Firebase Firestore SDK + Firebase Cloud Messaging	Live tracking, booking status, push notifications
Structured backend	Supabase Kotlin SDK (Postgrest, Auth, Storage)	Bookings, profiles, documents, payments, reviews
Auth	Firebase Auth (Email/Password, Google, Anonymous/Guest)	One login bridges to Supabase via JWT claim mapping
Images	Coil (Compose-native image loading)	Loads Cloudinary-hosted PNGs/illustrations efficiently with caching

Maps	OSMDroid (OpenStreetMap, fully free) — swappable to Google Maps SDK later via the <code>MAPS_PROVIDER</code> build flag	Zero-cost location/tracking to start
Video (splash)	Media3 ExoPlayer	Smooth, modern, officially maintained video playback for the splash screen
Build	Gradle Kotlin DSL, GitHub Actions for CI/CD	Produces the signed <code>.aab/.apk</code> automatically on push (see §10)

4. Icons — What’s Coded vs. What’s AI-Generated (decision rule)

Apply this rule to **every** icon need in §7 before deciding whether to write Kotlin or add an entry to `01_ai_images.json`:

1. **Is it a common UI glyph** (back arrow, search, bell, star, home, settings, calendar, phone, checkmark, location pin, camera, share, close, menu, edit, delete, filter, chevron)? → Use **Material Symbols** via `androidx.compose.material.icons.extended (Icons.Rounded.* / Icons.Outlined.*)`. This is Google’s real, official, free icon set — **never** substitute an emoji character for these.
2. **Is it a brand-specific badge, category illustration, onboarding artwork, or empty-state illustration** that doesn’t exist as a simple line glyph (e.g. “nurse kneeling beside patient,” “Elite crown medallion,” “empty wallet illustration”)? → This is a **PNG asset**, cannot reasonably be hand-coded as a vector, so it goes into `01_ai_images.json` (§1.8).
3. **Is it a simple geometric badge shape that Compose can draw** (a plain circle with a checkmark, a simple colored dot, a progress ring)? → Draw it directly in Compose with `Canvas/Icon + Modifier.background(CircleShape)` — do not waste an AI image slot on something 5 lines of Kotlin can produce.
4. **App launcher icon** → built from the founder-provided master logo PNG via Android Studio’s Image Asset Studio (adaptive icon: foreground + background layers) — never AI-generated, never redrawn.

5. Splash Screen & Video Integration

1. Implement Android 12+’s native **SplashScreen API** (core-splashscreen library) for the instant cold-start frame (shows the app icon on brand teal background for the few hundred milliseconds before the first Activity draws) — this satisfies Android’s own splash requirements and avoids a jarring blank white flash.
2. Immediately after, launch a dedicated `SplashVideoScreen` Composable that plays the founder-provided `splash_video.mp4` (from `res/raw/`) **full-screen**, edge-to-edge (hide system status/navigation bars for this screen only, restore them afterward), using **Media3 ExoPlayer** with `ResizeMode.RESIZE_MODE_ZOOM` or `FIT` (whichever the video’s own aspect ratio needs to fill the screen without distortion — inspect the actual video dimensions once provided and choose accordingly, defaulting to `FIT` centered on brand-teal `#0D4A44` letterbox bars if the video isn’t a perfect device-matching ratio).
3. Video plays exactly once, no loop, no visible player controls, no skip button needed if the video is short (≤ 3 –4 seconds is the expected length for a splash); if the founder’s video runs longer, add a subtle “Skip” text in the bottom-right corner after 2 seconds, safe-area-inset aware.
4. On video completion (or skip), navigate to the Onboarding flow (first-time users) or straight to Login/Home (returning users, checked via a saved “has completed onboarding” flag in `DataStore`).
5. The **app logo** (same master PNG as the launcher icon) must additionally appear, correctly scaled and centered, on: the Login screen header, the Sign-Up screen header, the “About Homigo Care” settings screen, and the email/notification templates the Admin Panel sends — Compose-side, load it once via a shared `AppLogo()` composable component so every placement is guaranteed identical, never a separately-cropped copy.

6. Authentication & OTP Flow (via the Admin Panel’s EmailJS SMTP)

The Android app itself never talks to EmailJS directly (EmailJS is a JavaScript/web SDK and its API keys must stay server-side). Instead:

1. The **Admin Panel project** (separate Vercel-hosted Next.js repo) holds the EmailJS SMTP credentials and exposes two API routes: `POST /api/otp/send` and `POST /api/otp/verify` (already listed in §1.5).
2. **Sign-up / login flow in this Android app:**
 - User enters email (and password, for Email/Password sign-up) → app creates the Firebase Auth account → app calls `POST /api/otp/send` with the email → Admin Panel generates a 6-digit code, stores it in a Supabase `otp_codes` table with a 5-minute expiry, and sends it via EmailJS SMTP.
 - App shows the existing 6-box OTP-entry UI (screen P-07) → user types the code → app calls `POST /api/otp/verify` → on success, the Admin Panel marks the Firebase user’s Supabase profile row `email_verified = true` and returns a success flag → app proceeds to Profile Setup.
 - “Continue with Google” and “Guest” sign-ins **skip this OTP step entirely** (Google accounts are already verified; Guest accounts have no email to verify yet and are prompted to add one later if they want to keep their data).
3. **Phone-number SMS OTP is intentionally not wired yet** (per the main architecture decision) — the phone-number field and its “Verify Code” UI still exist in the design (so the layout doesn’t need to change later) but are backed by this same email-based flow for now. When Twilio/Firebase Phone Auth is added later, only the Admin Panel’s `/api/otp/send` implementation needs to change — this Android app’s screens do not.
4. Password-reset follows the identical pattern: `POST /api/otp/send` with purpose: “password_reset”, same 6-box entry screen, then a new-password screen.

7. Complete Screen-by-Screen Blueprint

How to read this table: every screen inherits the Global Design System from §2 automatically — the “Notes” column only calls out anything screen-specific (e.g. an icon that needs an AI asset). “Flow” shows what comes before/after so the agent can wire NavGraph.kt correctly in one pass. Screens are grouped: **Shared** (everyone sees these first) → **Patient** (P-xx) → **Professional** (PR-xx). A single installed app contains both flows; which one a signed-in user sees is decided by their role column in Supabase users.

7.1 Shared — Splash, Onboarding, Auth, Role Selection (9 screens)

ID	Screen	Comes After → Leads To	Content & Key Components
S-01	Splash (video)	App cold start → Onboarding (first launch) or Login (returning)	Full-screen splash_video.mp4 per §5, no buttons Illustration il_onboarding_homecare.png (AI asset), headline “Verified care, at your doorstep,” Skip + Next
S-02	Onboarding 1/3	S-01 → S-03	Illustration il_onboarding_verified.png (AI asset), headline about the 3-phase verified professionals
S-03	Onboarding 2/3	S-02 → S-04	Illustration il_onboarding_safety.png (AI asset), headline about GPS Safe Zone + SOS, “Get Started” button
S-04	Onboarding 3/3	S-03 → S-05	App logo top-center, Email field, Password field, “Continue with Google” button (official Google branding button, not a custom recreation), “Continue as Guest” text button, “Sign Up” link
S-05	Login	S-04 / logout anywhere → Home (success) or S-06 (no account)	Name, Email, Password, Confirm Password, checkbox linking to Terms (fetched live from CMS §1.5), “Create Account”
S-06	Sign Up	S-05 → S-07	Two large tappable cards: “I need care” (patient icon) vs “I provide care” (professional/stethoscope icon) — this single fork decides the entire rest of the user’s experience
S-07	Role Selection	S-06 → S-08 (Patient) or PR-03 (Professional)	
S-08	OTP Verification	S-06/S-07 or “Forgot Password” → Profile Setup / New Password	6-box code entry, “Resend code” (60s cooldown), per §6
S-09	Forgot Password	S-05 → S-08 (reset variant) → New Password screen	Email field, “Send Reset Code”

7.2 Patient Flow — 32 Screens (P-01-P-32)

(P-01 is functionally the same as S-01/S-05 above for a patient account already signed in — listed here for numbering continuity with the source spec.)

ID	Screen	Flow	Content & Key Components
P-08	Who is this for?	After S-08 (patient) → P-09	3 cards: “Myself (Adult),” “My Child,” “Elderly Family Member” — determines the CNIC/guardian logic in the next step Full name, CNIC number + front/back photo upload (Cloudinary signed preset), Date of Birth, Gender — for Minor/Elderly variants, this collects the guardian’s CNIC instead, plus the patient’s own name/DOB
P-09	Personal Details	P-08 → P-10	Map picker (OSMDroid) with “Use my

P-10	Address & Emergency Contact	P-09 → P-11	current GPS,” house/street fields, one required emergency contact (name + phone)
P-11	Home Dashboard	P-10 (first time) / app resume → any category screen	Top: greeting + notification bell icon; large icon-based service grid (category badge PNGs from Batch-02 assets: Nurse, Doctor, Lab, Medicine, Equipment, Supply); voice-search mic button (orange, pulsing); “Book Emergency” red banner always visible above the fold
P-12	Service Category	P-11 → P-13	Selected category’s sub-options (e.g. under “Nurse”: Home Visit / Dedicated 8-12-24h / Companion / Navigator), each showing Budget vs Premium price chips
P-13	Browse Professionals	P-12 → P-14	List of verified professionals matching the category, each card: profile photo, name, badge icon (Certified/Excellence/Elite from Batch-03), rating stars, distance, price, “Book” button
P-14	Professional Profile	P-13 → P-15	Full bio, hospital affiliation (“Currently working at...”), badges, 5-category rating breakdown, reviews list (moderated), “Book Now”
P-15	Booking Details	P-14 → P-16 (if Health Passport sharing relevant) or P-18	Booking type selector (Instant/Scheduled/Recurring/Online), date/time picker, voice-or-text notes field, “Add to combined booking” toggle
P-16	Health Passport PIN Share	P-15 → P-17/P-18	PIN entry field the patient sets for this specific booking (or “Share Emergency Info Only” toggle), explanation text that access auto-revokes when the visit ends
P-17	Supply Pre-Request	P-16 → P-18	Optional: nurse-suggested supply list appears here mid-flow or is added by the patient upfront; each item shows estimated cost, “Approve Cost” checkbox per item
P-18	Booking Summary & Payment	P-17 → P-19	Price breakdown (base + surcharge if peak/emergency – any wallet cashback), payment method selector (JazzCash/Easypaisa/Card/Bank/COD — pulled from the Admin Panel’s active payment_accounts), for non-COD: shows the receiving account + a “Transaction ID” input + optional screenshot upload
P-19	Booking Confirmed	P-18 → P-11 (or auto to P-20 near visit time)	Success illustration (success_booking_confirmed.png, AI asset), booking reference number, “View Booking”
P-20	Live Tracking	Triggered near scheduled time / on “Visit Started” → P-21	Map (OSMDroid) with professional’s live location (Firestore presence/bookings_live stream), ETA, call/message buttons, Safe Zone radius overlay
P-21	Visit In Progress	P-20 → P-22	Status timeline (Arrived → In Progress → Completing), SOS button always visible (red, pinned, exempt from scroll per §2 mobile-safe rule)
P-22	Visit Complete	P-21 → P-23	Summary of what was done (from the professional’s mandatory visit notes), “Rate this visit” CTA
P-23	Rating & Review	P-22 → P-11	5-category star rating (Punctuality/Professionalism/Medical Skills/Communication/Overall), optional text review, “Post anonymously” toggle
			5 tabs: Basic Info, Medical History, Documents, Activity Log (auto-populated from completed bookings),

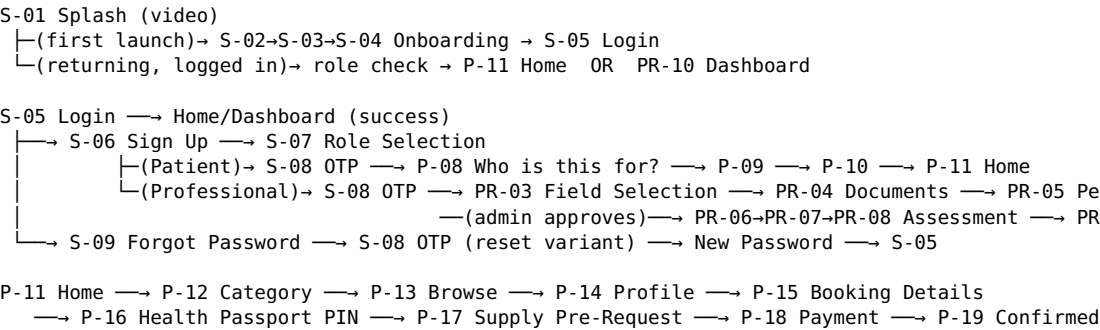
P-24	Health Passport	Accessible from bottom nav anytime	Vitals (line charts for BP/sugar/weight/temperature over time) — PIN-gate icon top-right to manage the Emergency PIN separately Giant red SOS circle (3-second hold to trigger, per the safety spec), one-tap numbers grid (Rescue 1122, Police 15, Edhi 115, Women’s Helpline 1099, Homigo Care Support), must render and function even with no internet (cache the numbers locally, SMS fallback for the SOS trigger itself)
P-25	Emergency Screen	Reachable within 2 taps from anywhere (bottom nav or floating action button)	Test/panel picker, partner lab selector, home-sample-collection toggle
P-26	Lab Test Booking	From P-12 (Lab category) → P-18	Prescription upload (Cloudinary) or “Link my Health Passport prescription,” pharmacy partner selector, delivery address confirm
P-27	Medicine Order	From P-12 (Medicine category) → P-18	Balance display, transaction history list, “Top Up” (shows cashback tiers per the pricing rules), referral code entry field
P-28	Wallet	Bottom nav →	User’s unique referral code + share button, live monthly leaderboard list, milestone progress bar (5/10/25 referrals)
P-29	Referral & Leaderboard	From P-28 or Profile menu	3 visually distinct priority tiers (color-coded left border: red=mandatory, orange=important, gray=optional), 30-day scrollable history
P-30	Notifications	Bottom nav (bell icon badge count)	Profile photo + name edit, saved addresses, saved payment methods (optional to add), language toggle (Urdu/English — full RTL-safe switch), notification quiet-hours setting, “About Homigo Care” (CMS-driven), Support contact (CMS-driven), Delete Account (30-day soft-delete flow), Logout
P-31	Profile & Settings	Bottom nav	Filterable list (Upcoming/Completed/Cancelled), tap any row → re-opens that booking’s summary/rating screen
P-32	Booking History	Bottom nav or Profile menu	

7.3 Professional Flow — 15 Screens (PR-01-PR-15)

ID	Screen	Flow	Content & Key Components
PR-03	Professional Field Selection	After S-07 → PR-04	Category cards (Nursing Division, Medical & Dental, Allied Rehab & Diagnostics, Pharmacy Operations, etc. — matches the founder’s own mockup screen exactly), then a second-level list of the specific role (Physiotherapist, GP, LHV, etc.) driven by the <code>role_requirements</code> Supabase table so adding a new role later needs zero app-code changes Dynamic form — for the selected role, render exactly the required documents from <code>role_requirements.required_documents</code> (e.g. Degree, License/Registration, Experience Letter, Affiliation Letter) plus the universal CNIC front/back — each field is a Cloudinary signed-upload tile with a live thumbnail preview
PR-04	Document Upload	PR-03 → PR-05	Status illustration, “Your documents are under review”
PR-05	Verification Pending	PR-04 → PR-06 (once admin approves, push notification	

		triggers this transition)	message, estimated review time, support contact (CMS-driven)
PR-06	Assessment Intro	Triggered after admin approval → PR-07	Explains the 3 phases (Basic Knowledge 30 MCQ/45min/70%, Practical Scenario 20Q/60min/75%, Ethics 20Q/30min/80%), single-active-device warning, “Start Phase 1”
PR-07	Assessment — Live Test	PR-06 → PR-08	Server-driven countdown timer, one question per screen, no back-navigation once answered, copy-paste disabled, progress dots at top
PR-08	Assessment Results	PR-07 → PR-09 (pass) or PR-06 retry (fail, after cooldown)	Score breakdown per phase, badge earned animation (Certified/Excellence/Elite — Batch-03 AI assets)
PR-09	Profile Activation	PR-08 → PR-10	Set availability hours, service radius, pricing tier confirmation (Budget/Premium), “Go Live” toggle
PR-10	Professional Dashboard	PR-09 → any screen below	Online/Offline toggle (top, prominent), today’s earnings summary, upcoming bookings list, quick stats (rating, completed visits)
PR-11	Incoming Booking Request	Push-triggered overlay/screen → PR-12 (accept) or back to PR-10 (reject/timeout)	Patient’s booking details, 10-minute countdown ring , Accept/Reject buttons — auto-cascades to next professional after 3 rejections platform-wide (handled server-side, not in this screen’s logic)
PR-12	Active Booking	PR-11 → PR-13	“Start Visit” (activates the patient’s Safe Zone GPS tracking), navigation-to-patient map, call/message patient, view shared Health Passport PIN field if the patient provided one
PR-13	Visit Notes (mandatory)	PR-12 → PR-14 cannot be skipped	Structured form: vitals recorded, care provided (checklist + free text), supply items used (feeds \$1.7’s supply pre-request approval loop), photo attachment if relevant, “Complete Visit” (this is the record that later populates the patient’s P-24 Activity Log)
PR-14	Earnings & Payouts	Bottom nav	Available balance, payout request (Daily/Weekly/Monthly/On-demand, min PKR 500 — matches \$7 of the main architecture doc), transaction history, bank/JazzCash/Easypaisa account entered once for payout
PR-15	Professional Profile Management	Bottom nav	Edit bio/photo, view own badges, manage availability calendar, re-take assessment (if prompted by admin), Support contact (CMS-driven), Logout

8. App-Wide Navigation Map (text flow, for NavGraph.kt)



—(near visit time)—→ P-20 Live Tracking → P-21 In Progress → P-22 Complete → P-23 Rate

Bottom nav (always reachable from P-11/anywhere): Home · Health Passport (P-24) · Emergency (P-25, ≤2 taps) · Wallet (P-28) · Profile (P-31)

Secondary reachable: P-26 Lab, P-27 Medicine (from category grid), P-29 Referral (from Wallet/Profile), P-30 Notifications (bell icon), P-32 History (Profile menu)

PR-10 Dashboard → PR-11 Incoming Request → PR-12 Active Booking → PR-13 Visit Notes (mandatory) → PR-10

Bottom nav: Dashboard · Earnings (PR-14) · Profile (PR-15)

Implement this exactly as one NavHost with two nested graphs (patientGraph, professionalGraph) both reachable from a shared authGraph — this keeps the single-APK, dual-role structure clean in code and matches how role is checked once at login rather than scattered through the app.

9. Data Sync Architecture (Android-side summary)

Data Type	Source	Android Reads Via	Android Writes Via
Login session	Firebase Auth	FirebaseAuth.currentUser	Firebase Auth SDK calls
Live GPS / booking status / presence / chat	Firebase Firestore	Real-time snapshotListener (Flow-wrapped)	Firestore set/update
Push notifications	Firebase Cloud Messaging	FirebaseMessagingService	(Admin Panel triggers sends server-side)
Profiles, bookings ledger, reviews, payments, documents metadata, role requirements, wallet	Supabase Postgres	Supabase Kotlin SDK (Postgrest.from("table").select()) with RLS auto-scoping to the logged-in user	Supabase insert/update calls, always going through RLS
CNIC/degree/prescription/receipt files	Cloudinary (signed)	Fetch signed URL from Admin Panel API before display	Upload via signed preset after requesting a signature
App icon/illustration assets	Local drawable resources (preferred) or Cloudinary homigo_public_assets (if founder chooses to host them remotely instead of bundling)	painterResource() or Coil AsyncImage	N/A — assets are placed by the founder per §1.8, not generated at runtime
OTP send/verify, CMS content (support info, About Us, FAQ, legal, pricing)	Admin Panel Vercel API	Retrofit calls to ADMIN_API_BASE_URL	N/A (read-only from the app’s side, except triggering OTP send)

This table is the single reference for “which SDK do I call for this piece of data” — the agent should not invent a new data path outside this table without updating it first.

10. Build & Release Pipeline

- .github/workflows/android-release.yml triggers on every push to main:
 - Checkout → set up JDK 17 → ./gradlew testDebugUnitTest → ./gradlew bundleRelease (and assembleRelease for a sideloadable .apk during testing).
 - Sign the release build using a keystore stored in GitHub Actions **encrypted secrets** (never committed to the repo).
 - Upload the resulting .aab/.apk as a workflow artifact; optionally add a Play Store internal-track upload step once the founder has a Play Console account.
- Versioning: bump versionCode/versionName in app/build.gradle.kts on every release-worthy push; keep a CHANGELOG.md at the repo root.
- homigo-care-dev Firebase project + a debug build variant point to it automatically via Gradle build-type-specific google-services.json/flavor config, so testing never touches production data — this also keeps the **zero-fake-data rule** intact in production while still allowing free experimentation in dev.

11. Final QA Checklist Before Calling the Build “Done”

- ☐ Every screen in §7 uses only colors/fonts/shapes from core/Theme.kt — zero hardcoded hex values anywhere in ui/.
- ☐ Every icon is either a Material Symbol, a Compose-drawn shape, or a founder-approved AI PNG placed via §1.8 — no emoji characters used as icons anywhere.
- ☐ Splash video plays full-screen exactly once, correct aspect handling, then routes correctly for first-time vs. returning users.
- ☐ App logo appears identically (same asset, same composable) on Login, Sign Up, About screen, and matches the launcher icon.
- ☐ No screen clips content on a small device (360×640) or a large device (430×932) — test both per the mobile-safe-area rules.
- ☐ SOS button (P-25) is reachable within 2 taps from every screen and works with cached emergency numbers even with no network.
- ☐ Health Passport (P-24) enforces the PIN gate for professional access and logs every access.

- ☐ Payment screen (P-18) correctly lists only the **currently active** `payment_accounts` from Supabase — never a hardcoded account number.
- ☐ No table/collection in §9 contains seed/demo data — every list renders a genuine empty state on a fresh install pointed at an empty backend.
- ☐ `assets_pending/01_ai_images.json` exists, is valid JSON, has between 20–50 entries, and every entry’s `destination_path` matches a real reference used somewhere in the Compose code (no orphaned requests, no missing references).
- ☐ OTP send/verify round-trips correctly against the Admin Panel’s `/api/otp/*` endpoints in a real end-to-end test (not mocked).
- ☐ Role fork at S-07 correctly routes to the entire Patient graph or the entire Professional graph with no shared-state leakage between the two.